

Lab 4 – VibeCoding
Fruit Ninja Pygame
Chat / LLM History

Git repo - https://github.com/NitheshMogaveer777/41_fruit-ninja

Purpose: Record of the LLM-assisted work for Tasks 1–4.

User – Initial request

give me how to solve step wise

Assistant

Explained the Lab 4 workflow: set up Python/Pygame, run the original project, record the before video, complete Tasks 1–4 one at a time, make a separate Git commit for each task, record the after video, export chat history, and push the project.

User – LLM prompt question

where i have to use llm prompt

Assistant

Explained that the prompts should be used in ChatGPT, Claude, or Gemini, while code changes are made in VS Code in the same existing project folder. Recommended keeping one conversation so the complete LLM history can be exported.

User – Task 1

I am completing Task 1 of my Fruit Ninja Pygame lab.

The current collision detection only checks whether the latest mouse position is inside the fruit. Fast mouse swipes can jump over a fruit.

Modify the code so that the line segment between the previous mouse position and current mouse position is checked against the fruit's circular collision area.

Keep the existing project structure and behavior.

Give me the exact code changes required for fruit.py and game_engine.py.

Do not add unrelated features.

Assistant – Task 1

Recommended changing fruit.py from single-point collision to a contains_segment method. The method calculates the closest point on the mouse-motion line segment to the fruit center and checks the distance against the fruit radius. game_engine.py was changed so _handle_motion uses the previous trail position and current mouse position to test the complete segment.

Test with: python main.py.

Commit: git add game/fruit.py game/game_engine.py

git commit -m "Fix fast swipe collision detection".

User

i have to do it in the same folder or different

Assistant

Explained that all tasks must be done in the same existing Fruit Ninja project folder, not in separate Task folders.

User – Task 2

Now implement Task 2 only.

When the player slices a bomb or loses all lives, display a proper Game Over screen inside the Pygame window.

Show:

- GAME OVER

- Final Score
- A message telling the player how to continue

The game should stop normal gameplay while the Game Over screen is shown and wait for keyboard input.

Do not implement replay or sound yet.
Keep the existing code structure.
Give me the exact code changes.

Assistant – Task 2

Recommended adding a Game Over font, stopping normal mouse gameplay after `game_over` becomes true, accepting keyboard input, and replacing the console-only message with a Pygame overlay showing GAME OVER, Final Score, and a continuation message.

Commit: `git add game/game_engine.py`
`git commit -m "Add game over screen".`

User – Task 3

Now implement Task 3 only.

After Game Over, display a replay menu with:

1. Easy
2. Medium
3. Hard
4. Exit

Each difficulty should change the fruit spawn rate and bomb probability.

Easy = slower and fewer bombs
Medium = default difficulty
Hard = faster spawning and more bombs

When the player chooses a difficulty, reset:

- score
- lives
- fruits
- `game_over` state

Then start a new game.

Do not add sound yet.
Keep the existing functionality.
Give me the exact code changes.

Assistant – Task 3

Recommended adding a replay menu controlled by keys 1–4 and a `start_new_game(difficulty)` method.

Difficulty values:

Easy: spawn interval 70, bomb chance 0.08, speed scale 0.9.
Medium: spawn interval 55, bomb chance 0.15, speed scale 1.0.
Hard: spawn interval 40, bomb chance 0.25, speed scale 1.1.

The new-game method resets lives, score, fruits, trail, spawn timer, and game-over state. `main.py` was updated only to exit when `exit_requested` is true.

Commit:
`git add game/game_engine.py main.py`
`git commit -m "Add replay difficulty options".`

User – Task 3 error

NameError: name 'continue_text' is not defined

Assistant – Error fix

Explained that an old Task 2 `continue_text` reference remained in `render()`. Recommended removing only the obsolete `continue_text` drawing block while keeping the GAME OVER and Final Score blocks and adding the new difficulty menu.

User

i have to delete this thing

Assistant

Explained that the GAME OVER and Final Score `screen.blit` blocks must be kept. Only the obsolete `continue_text` code should be removed.

User – Task 4

Now implement Task 4 only.

Add Pygame sound effects for:

1. Successfully slicing a fruit
2. Slicing a bomb
3. Game over

Create a sounds folder and load the WAV files safely.

Play the appropriate sound at the correct event.

If a sound file is missing, the game should not crash.

Do not change the existing game logic or difficulty system.

Give me the exact code changes and explain where each sound file should be placed.

Assistant – Task 4

Recommended creating `sounds/slice.wav`, `sounds/bomb.wav`, and `sounds/game_over.wav`. Added `os` import, `_load_sounds()`, and `_play_sound()`. Modified `_slice()` to play the fruit or bomb sound, modified the lives-zero Game Over condition to play the Game Over sound, and reset `game_over_sound_played` when starting a new game. Sound loading and playback were made safe so missing sound files do not crash the game. Difficulty settings remain unchanged.

User – Task 4 error

AttributeError: 'GameEngine' object has no attribute '_load_sounds'. Did you mean: '_play_sound'?

Assistant – Error fix

Explained that `self._load_sounds()` was called in `__init__` but the method was not correctly defined inside `GameEngine`. Recommended placing `_load_sounds()` and `_play_sound()` inside the class, immediately before `spawn_fruit()`, with correct indentation, and ensuring `import os` is present.

User

give sounds folder three files with sounds

Assistant

Created a ZIP containing three WAV files: `slice.wav`, `bomb.wav`, and `game_over.wav`. These should be extracted into a `sounds/` folder in the project root.

User

give me chat-history.pdf